

Building a Location-Aware Weather and Air Quality App

JavaScript, Geolocation, and External APIs

The Project

How can a simple browser application determine where a user is and automatically deliver useful environmental information for that location?

I built the Weather & Air Quality App to explore that problem using JavaScript and external APIs. The application detects a user's geographic coordinates and retrieves current weather and air-quality information for that location, including temperature, sunrise and sunset times, Air Quality Index (AQI), PM2.5, and PM10 measurements.

The project demonstrates a fundamental pattern in modern web development: combining multiple external data services into a single user experience. Instead of requiring users to manually enter a city or ZIP code, the application first determines location and then uses those coordinates to request relevant environmental data.

From Location to Environmental Data

1. Obtain the user's latitude and longitude.
2. Use those coordinates to request local weather data.
3. Request air-quality data for the same location.
4. Process the responses in JavaScript.
5. Present the resulting information in a readable browser interface.

For primary location detection, the application uses the browser's Geolocation API. If browser-based geolocation is unavailable, the Google Geolocation API can serve as an alternative method for obtaining geographic coordinates. Once latitude and longitude are available, the application sends them to two Open-Meteo services. The Weather API provides current temperature and sunrise and sunset information; the Air Quality API provides U.S. AQI, PM2.5, and PM10 data. JavaScript connects these pieces: location becomes input to the API requests, the APIs return environmental data, and the application processes those responses for presentation to the user.

Working with Multiple APIs

One of the most useful lessons from the project was understanding that API integration involves more than simply retrieving data. Each service has a specific role. Browser geolocation or Google's Geolocation API answers "Where is the user?" The weather API answers "What are the current weather conditions at that location?" The air-quality API answers "What is the current air quality there?"

The application therefore creates a small data pipeline in which the output of one process—the user's coordinates—becomes the input required by other services. This architecture also illustrates why APIs are so useful in web development. The application does not need its own weather stations, environmental sensors, or meteorological database. Instead, it combines specialized services and focuses on retrieving, processing, and presenting the information users need.

Turning API Output into Useful Information

Raw API data is not necessarily meaningful to every user. Air quality is a good example. A numerical AQI value becomes considerably more useful when accompanied by an understandable category. The application therefore interprets the returned U.S. AQI value using categories ranging from Good and Moderate through Unhealthy, Very Unhealthy, and Hazardous. The interface also displays PM2.5 and PM10 measurements.

This distinction between retrieving data and communicating information is important. An API can provide the underlying values, but the application still has to decide how those values should be organized and presented so users can understand them quickly.

Implementation Considerations

The project uses HTML5 and CSS3 for structure and presentation, with vanilla JavaScript handling application logic and API communication. API requests are implemented using XMLHttpRequest.

Location-aware applications introduce an additional consideration: user permission. Browser geolocation depends on the user allowing location access, and behavior can vary among browsers. Providing an alternative geolocation method offers one way of thinking about this dependency. It also introduces another issue: credentials.

The Google Geolocation API requires an API key. Although a key can be inserted into JavaScript for a learning project, private API credentials should not be committed to a public repository. A production implementation should use an appropriate secure configuration or server-side approach rather than exposing sensitive credentials in client-side source code.

Limitations and Next Steps

The application works best in Firefox and Chrome, while Safari may encounter geolocation issues in some cases. It also depends on an active internet connection and the availability of the external APIs.

A future version could add seven-day forecasts, weather-condition descriptions, saved locations, support for comparing multiple locations, or weather alerts. Error handling could also be expanded to give users clearer feedback when location permission is denied or an external API is unavailable. Moving from XMLHttpRequest to a more modern asynchronous JavaScript approach would also be a logical direction for a future iteration.

What I Learned

I originally developed the Weather & Air Quality App as a JavaScript course project at Mission College. Building it gave me practical experience connecting JavaScript to external APIs, working with browser geolocation, processing returned data, and presenting location-specific information through a web interface.

More importantly, the project reinforced a broader software-development concept: even a relatively small application can depend on several systems working together. The browser supplies location, external services supply environmental data, JavaScript connects the components, and the interface turns the results into information a user can act on.

Documenting the project also required looking at the application from another developer's perspective: explaining not just what the code does, but how the components fit together, why each service is needed, where limitations exist, and what could be improved next.

Repository: github.com/powerbar/weather-air-quality-app

Live application: <https://powerbar.github.io/weather-air-quality-app/>